



Principles of Pipelining

Basic Computer Architecture

Outline

- * Overview of Pipelining
- * A Pipelined Data Path
- * Pipeline Hazards
- * Pipeline with Interlocks
- * Forwarding
- * Performance Metrics
- * Interrupts/ Exceptions



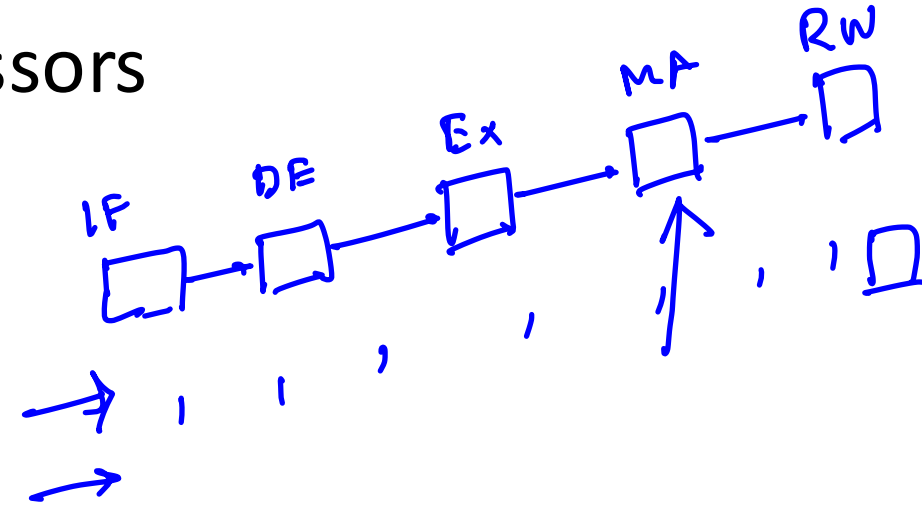
mop

Up till now

- * We have designed a processor that can execute all the SimpleRisc Instructions
- * We have look at two styles :
 - * With a hardwired control unit ✓
 - * Microprogrammed control unit ✓
 - * Microprogrammed data path
 - * Microassembly Language
 - * Microinstructions

Designing Efficient Processors

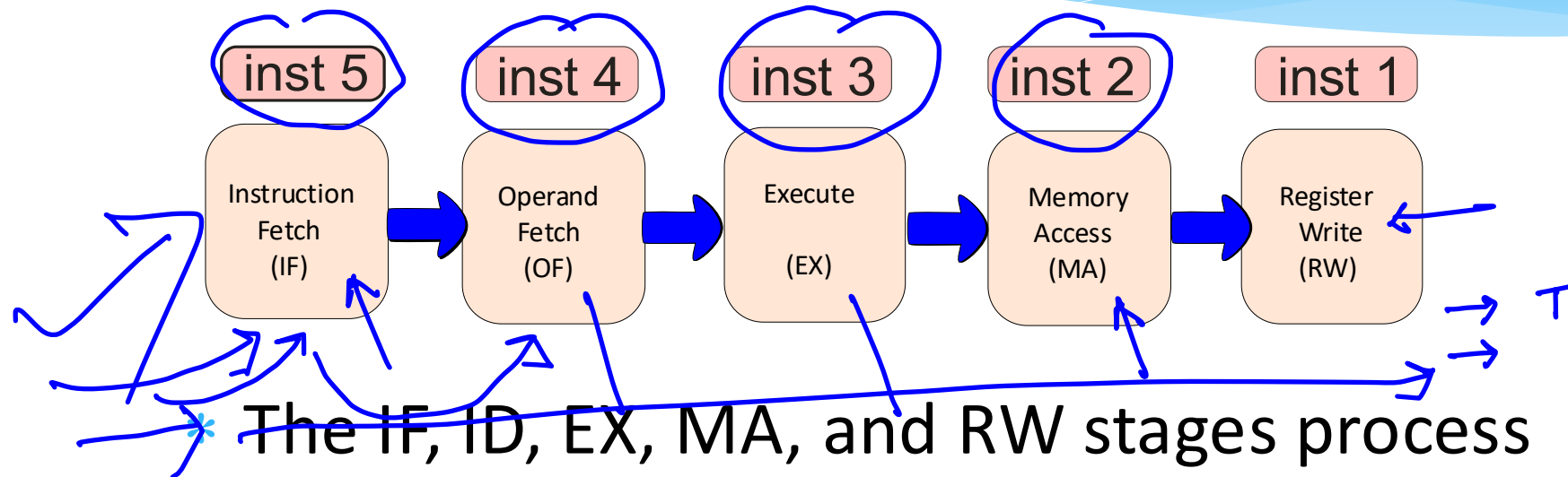
- * **Microprogrammed** processors are much slower than **hardwired** processors
- * Even **hardwired** processors
 - * Have a lot of **waste** !!!
 - * We have 5 stages.
 - * What is the IF stage doing, when the MA stage is active ?
 - * **ANSWER** : It is **idling**



The Notion of Pipelining

- * Let us go back to the **car assembly line**
 - * Is the **engine shop** idle, when the **paint shop** is painting a car ?
 - * **NO** : It is building the engine of another car
 - * When this engine goes to the body shop, it builds the engine of another car, and **so on**
 - * **Insight** :
 - * **Multiple cars** are built at the same time.
 - * A car **proceeds** from one stage to the next

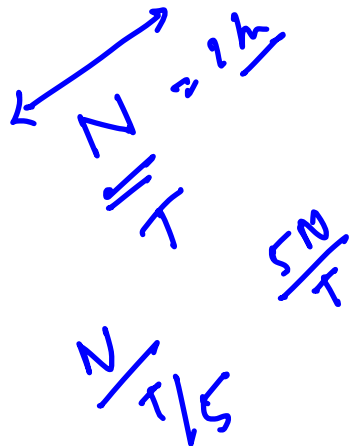
Pipelined Processors



5 instructions simultaneously

* Each instruction proceeds from one stage to the next

* This is known as **pipelining**

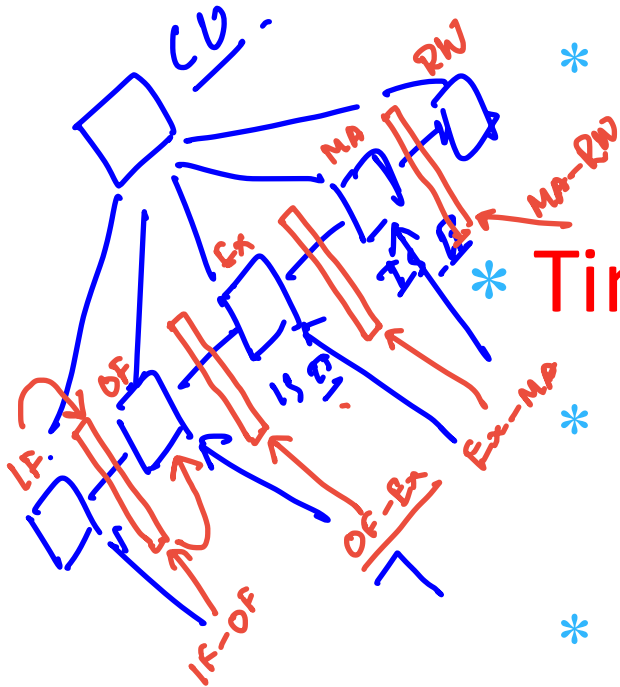


Advantages of Pipelining

- * We keep all parts of the data path, busy all the time
- * Let us assume that all the 5 stages do the same amount of **work**
 - * **Without** pipelining, every **T** seconds, an instruction completes its **execution**
 - * **With** pipelining, every **T/5** seconds, a new instruction completes its **execution**

Design of a Pipeline

* Splitting the Data Path

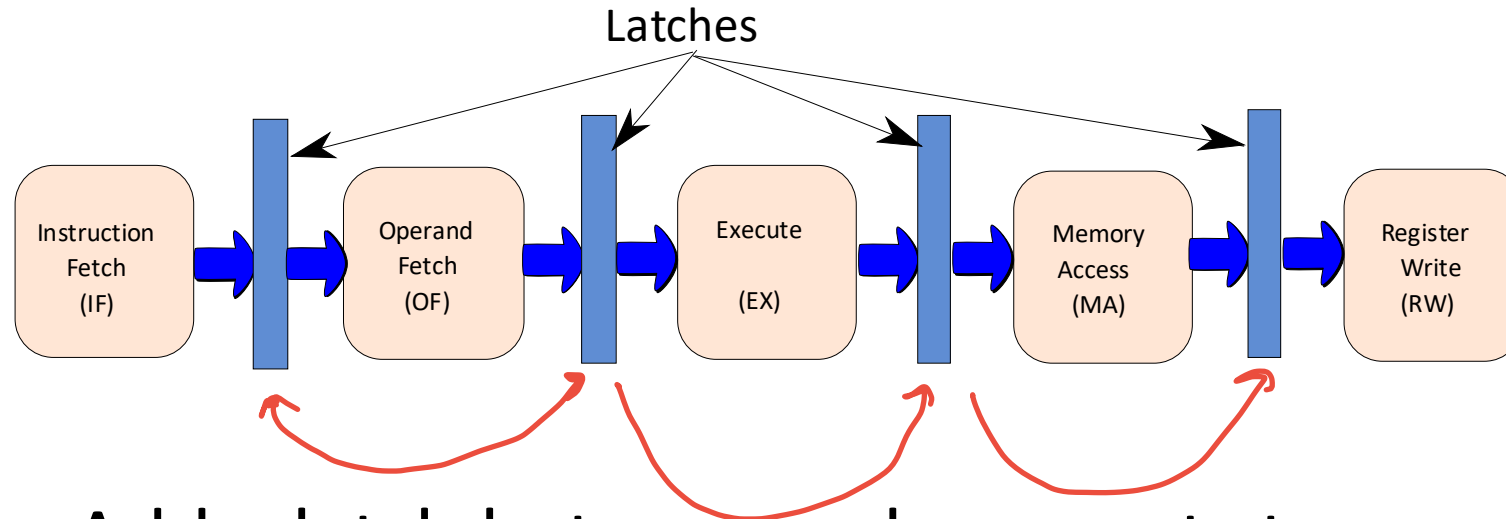


- * We divide the data path into 5 parts : IF, OF, EX, MA, and RW

* Timing

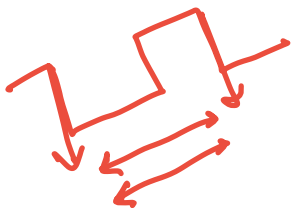
- * We insert latches (pipeline registers) between consecutive stages
- * 4 Latches → IF-OF, OF-EX, EX-MA, and MA-RW
- * At the negative edge of a clock, an instruction moves from one stage to the next

Pipelined Data Path with Latches



- * Add a latch between subsequent stages.

- * Triggered by a negative clock edge

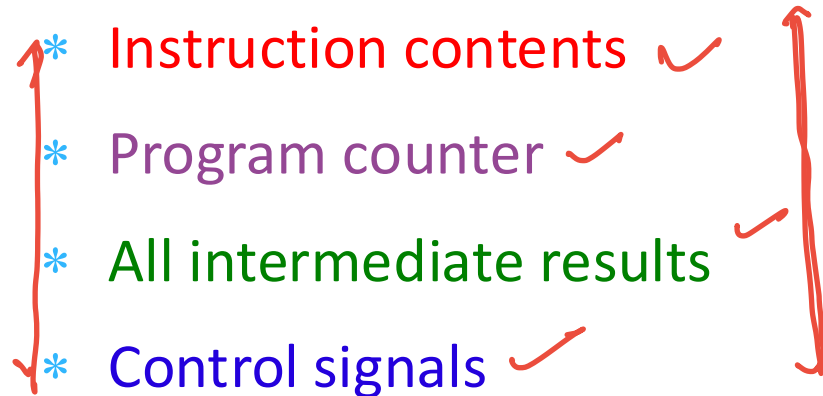


The Instruction Packet

- * What travels between stages ?

- * **ANSWER** : the instruction packet

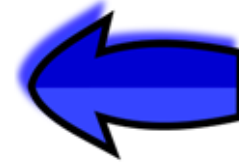
- * Instruction Packet

- * Instruction contents ✓
 - * Program counter ✓
 - * All intermediate results ✓
 - * Control signals ✓
- 
- A diagram consisting of two red vertical brackets. The left bracket groups the first three items: 'Instruction contents', 'Program counter', and 'All intermediate results'. The right bracket groups the last two items: 'All intermediate results' and 'Control signals'.

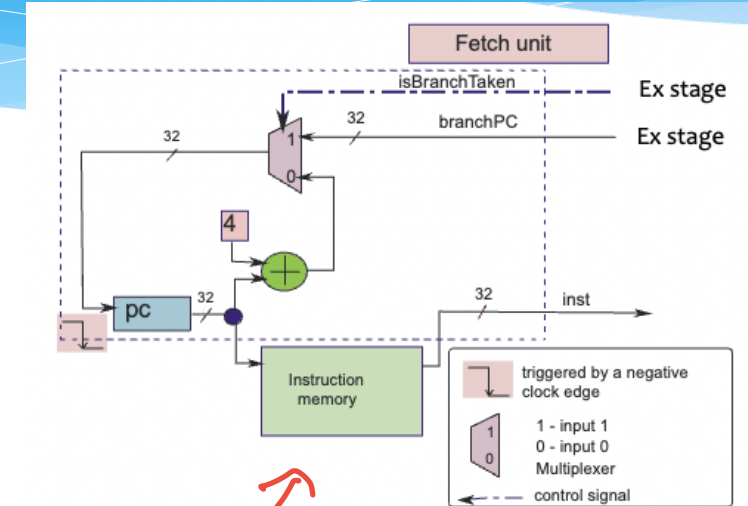
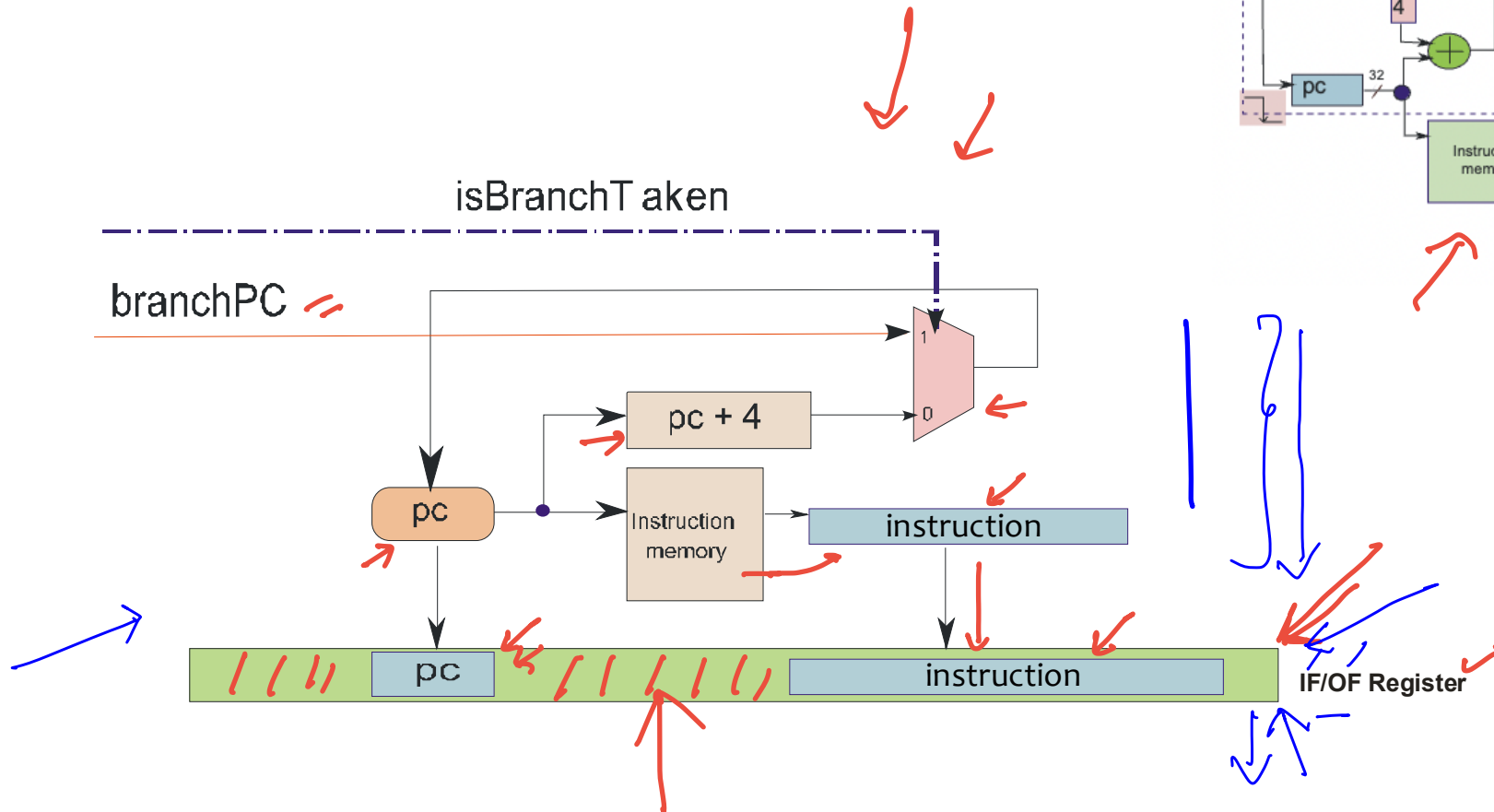
- * Every instruction moves with its entire state, no interference between instructions

Outline

- * Overview of Pipelining
- * A Pipelined Data Path
- * Pipeline Hazards
- * Pipeline with Interlocks
- * Forwarding
- * Performance Metrics
- * Interrupts/ Exceptions

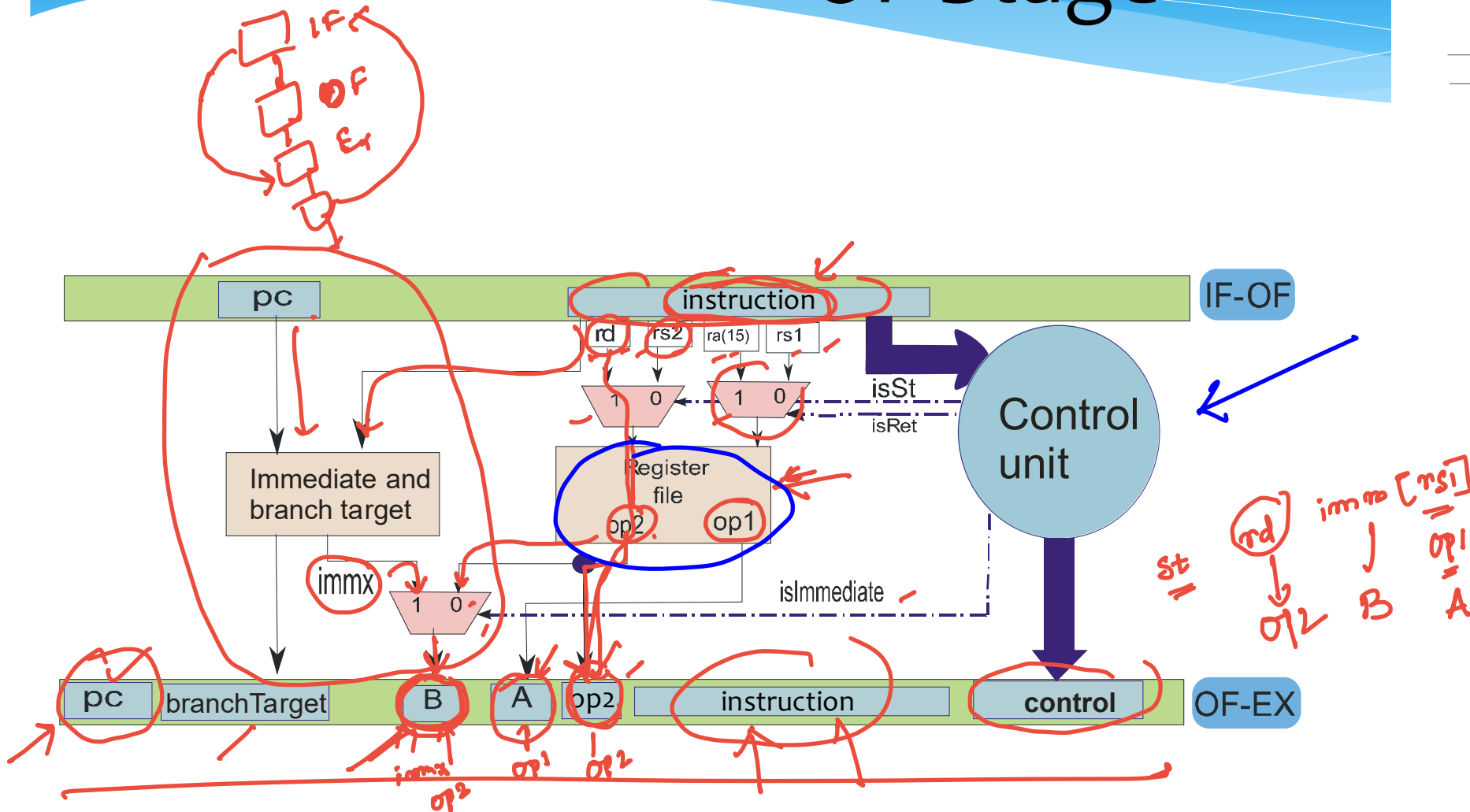
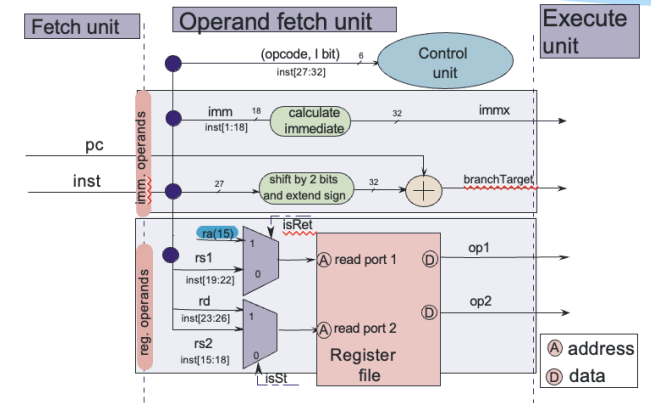


IF Stage



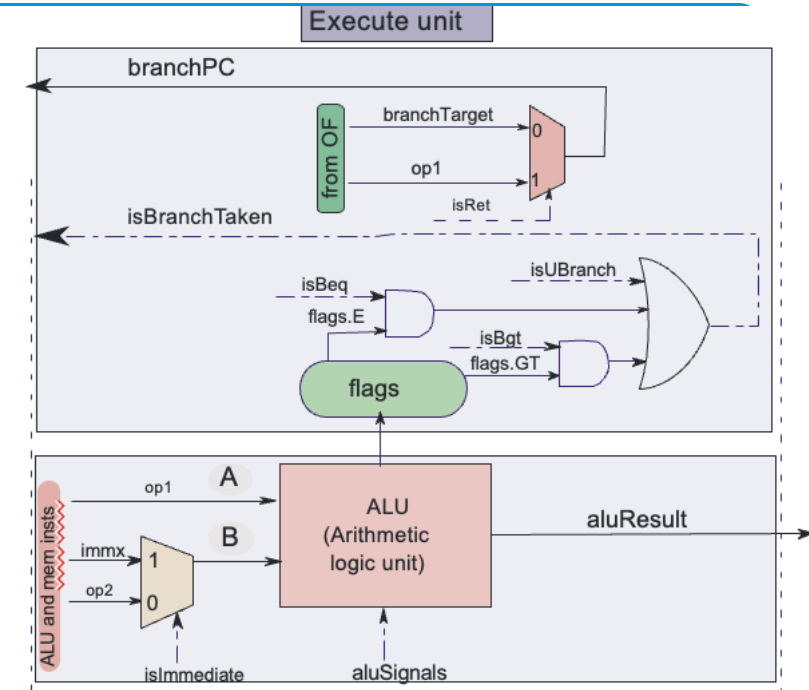
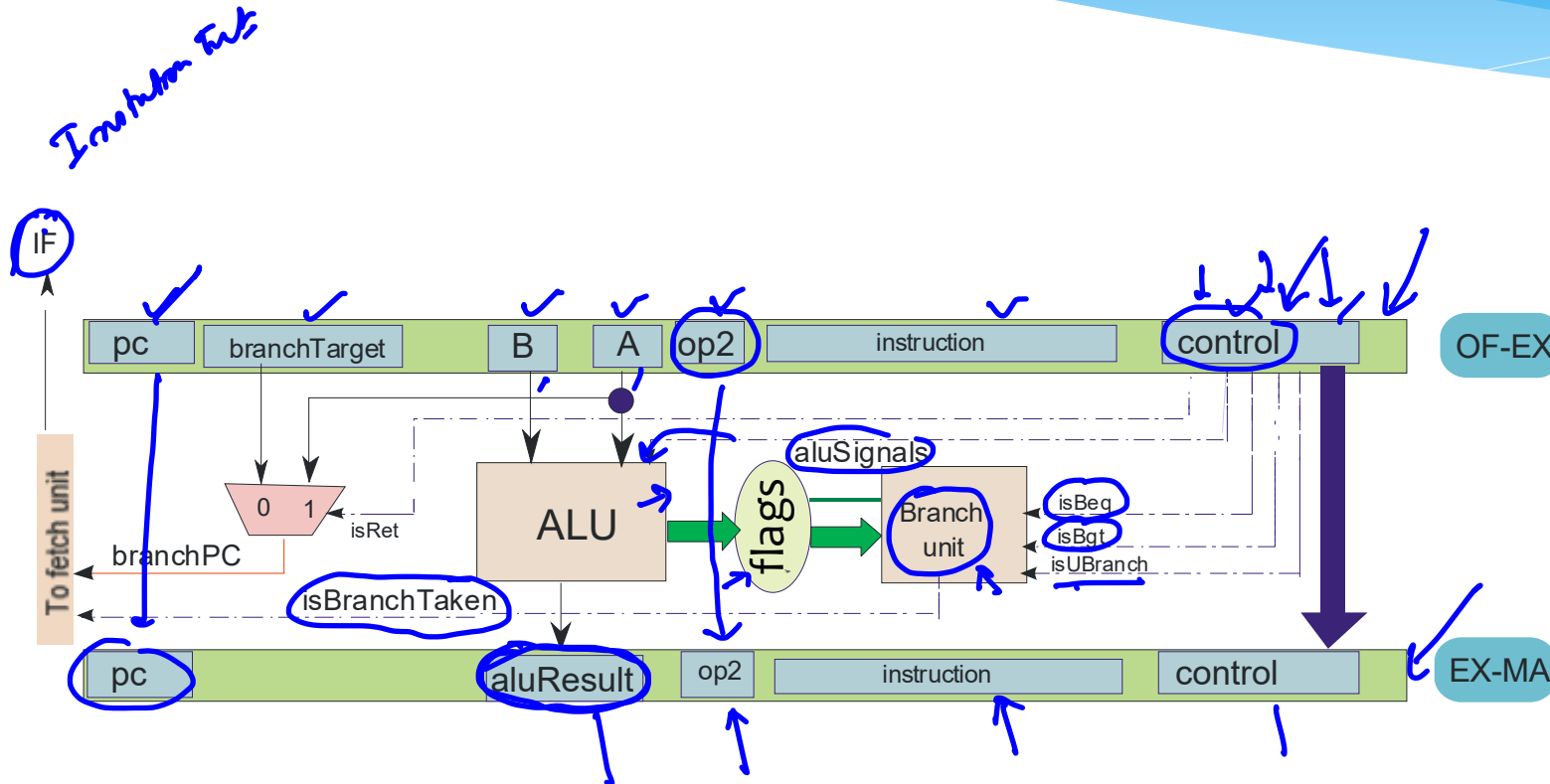
* Instruction contents saved in the **instruction** field

OF Stage



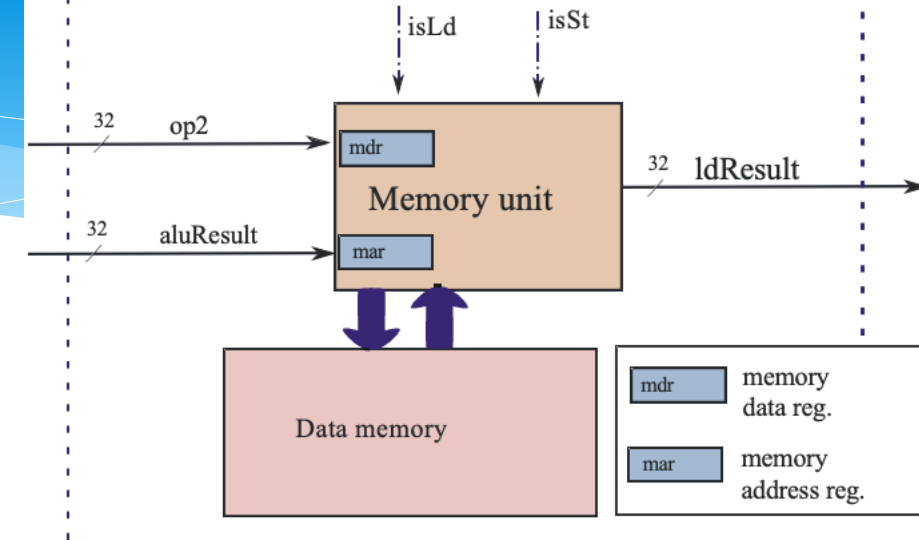
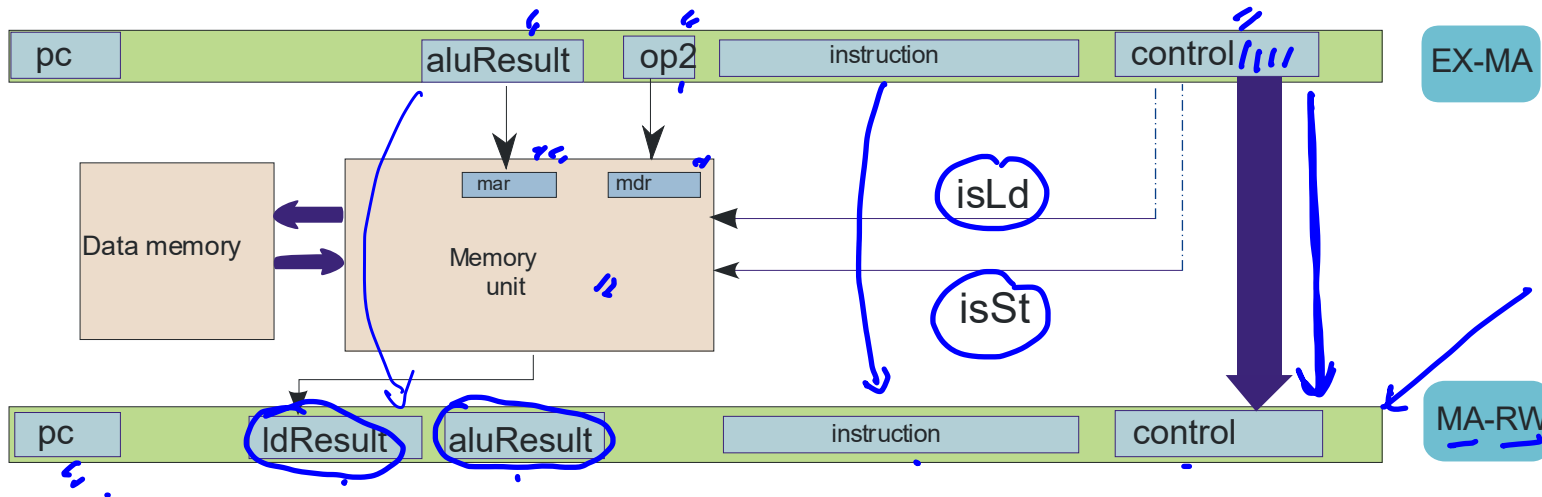
* **A, B** → ALU Operands, **op2** (store operand),
control (set of all control signals)

EX Stage



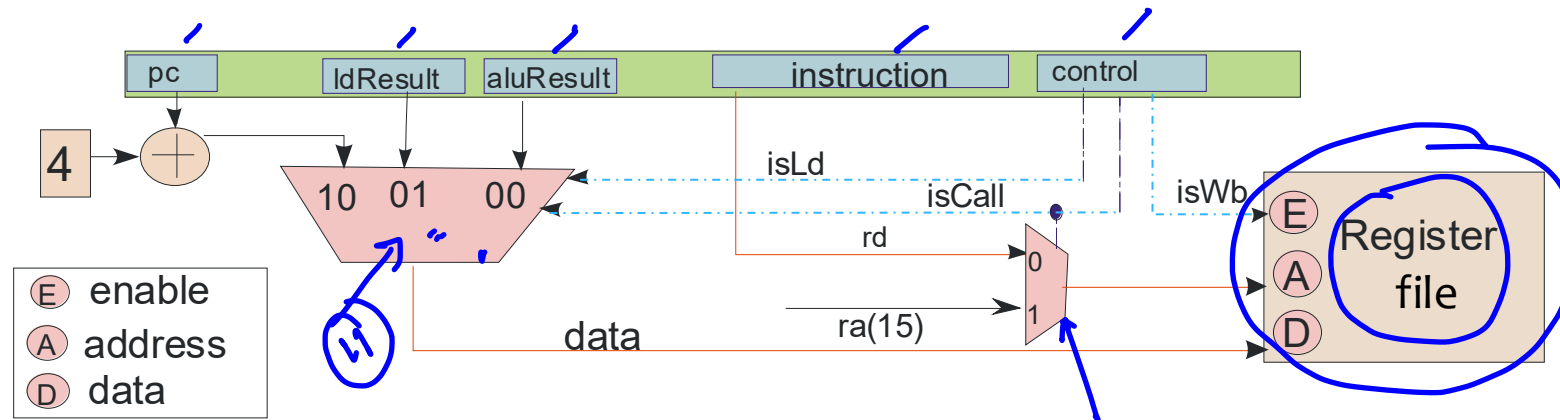
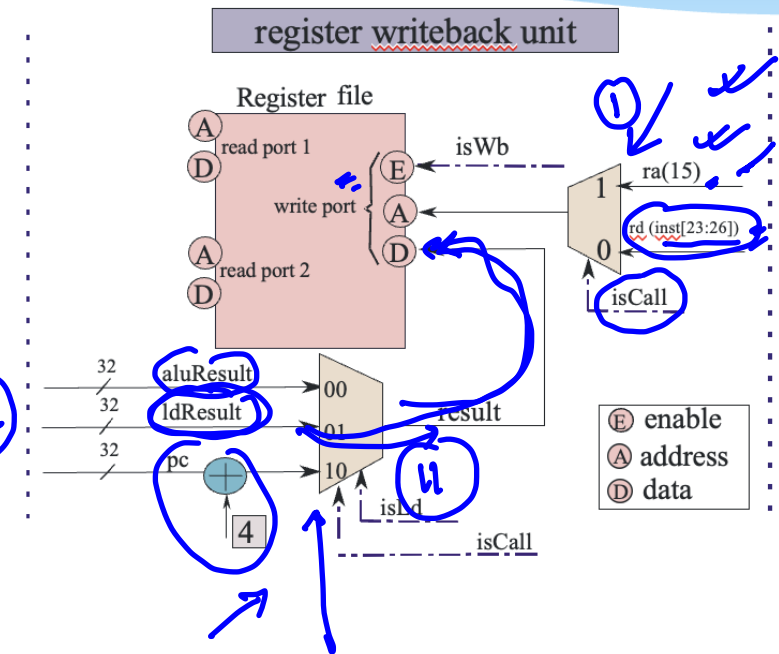
- * **aluResult** → result of the ALU Operation
- * **op2, control, pc, instruction** (passed from OF-EX)

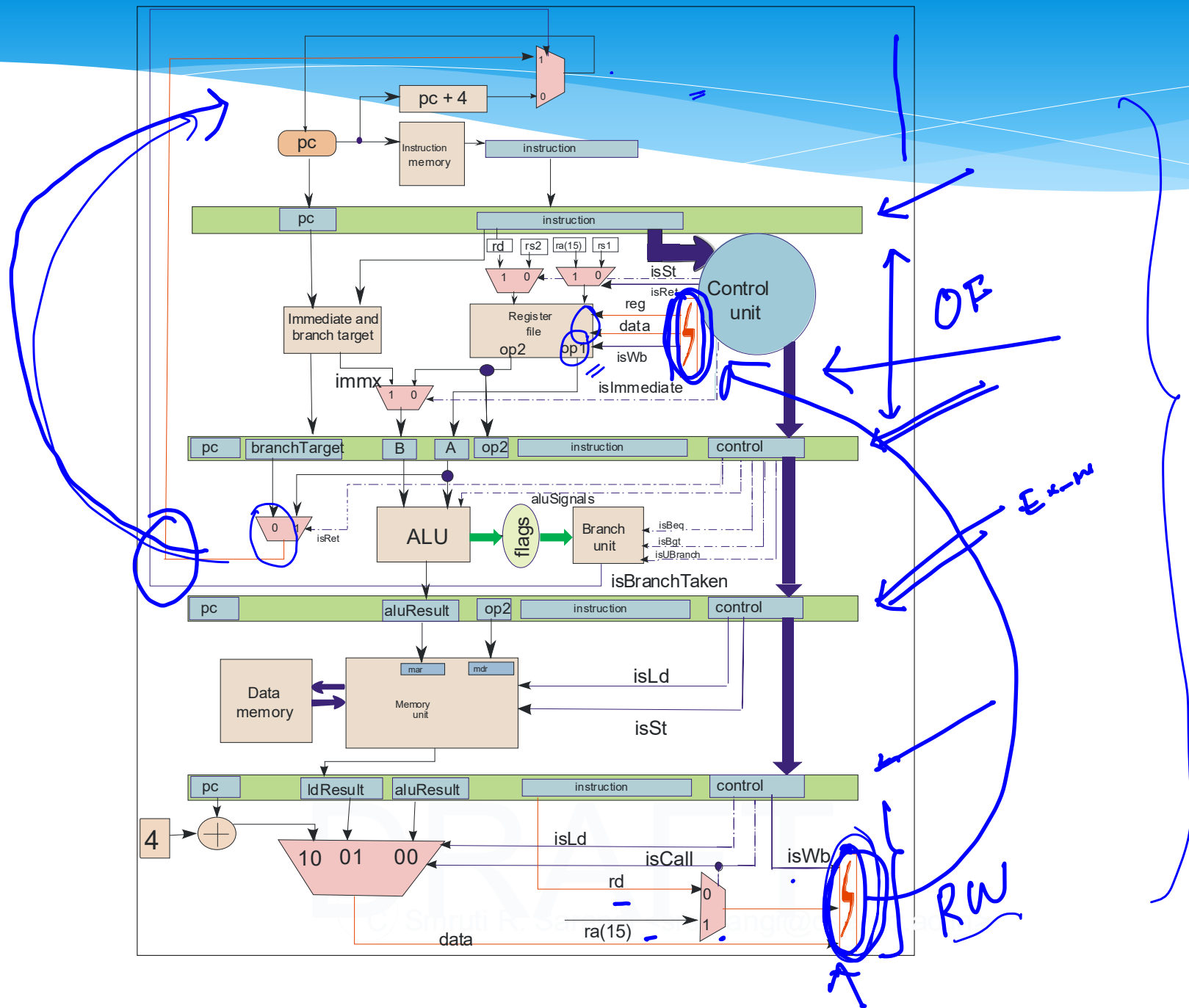
MA Stage



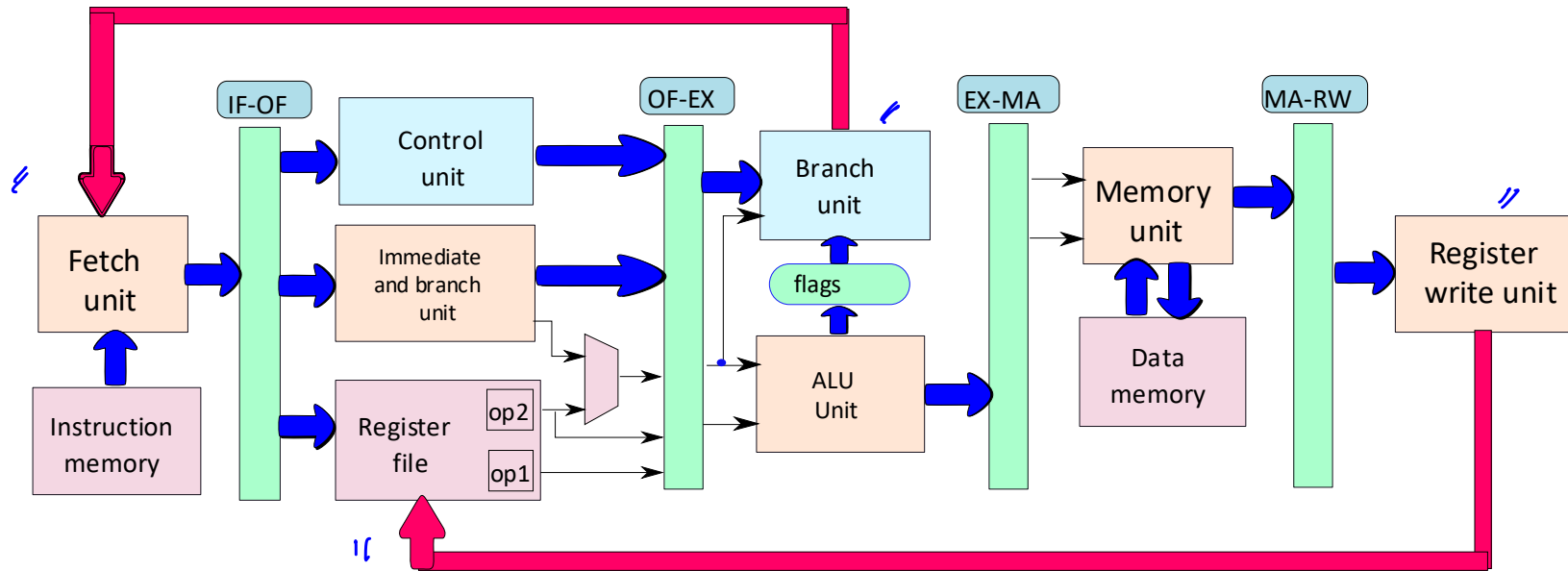
- * **IdResult** → result of the load operation
- * **aluResult, control, pc, instruction** (passed from EX-MA)

RW Stage





Abridged Diagram



Outline

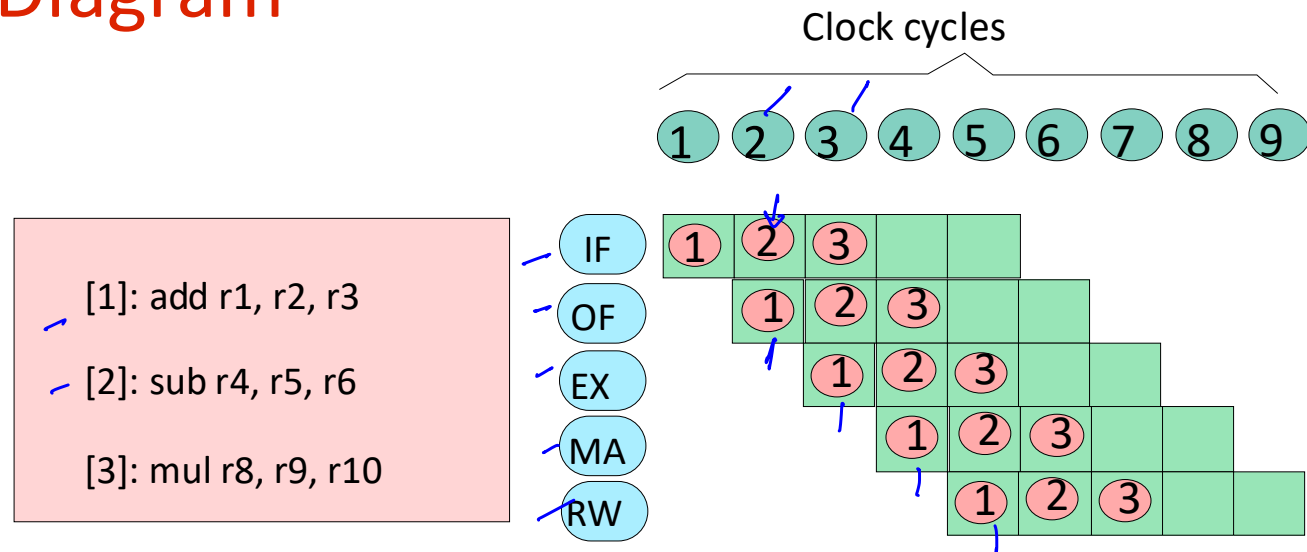
- * Overview of Pipelining
- * A Pipelined Data Path
- * Pipeline Hazards
- * Pipeline with Interlocks
- * Forwarding
- * Performance Metrics
- * Interrupts/ Exceptions



Pipeline Hazards

- * Now, let us consider correctness
- * Let us introduce a new tool → Pipeline Diagram

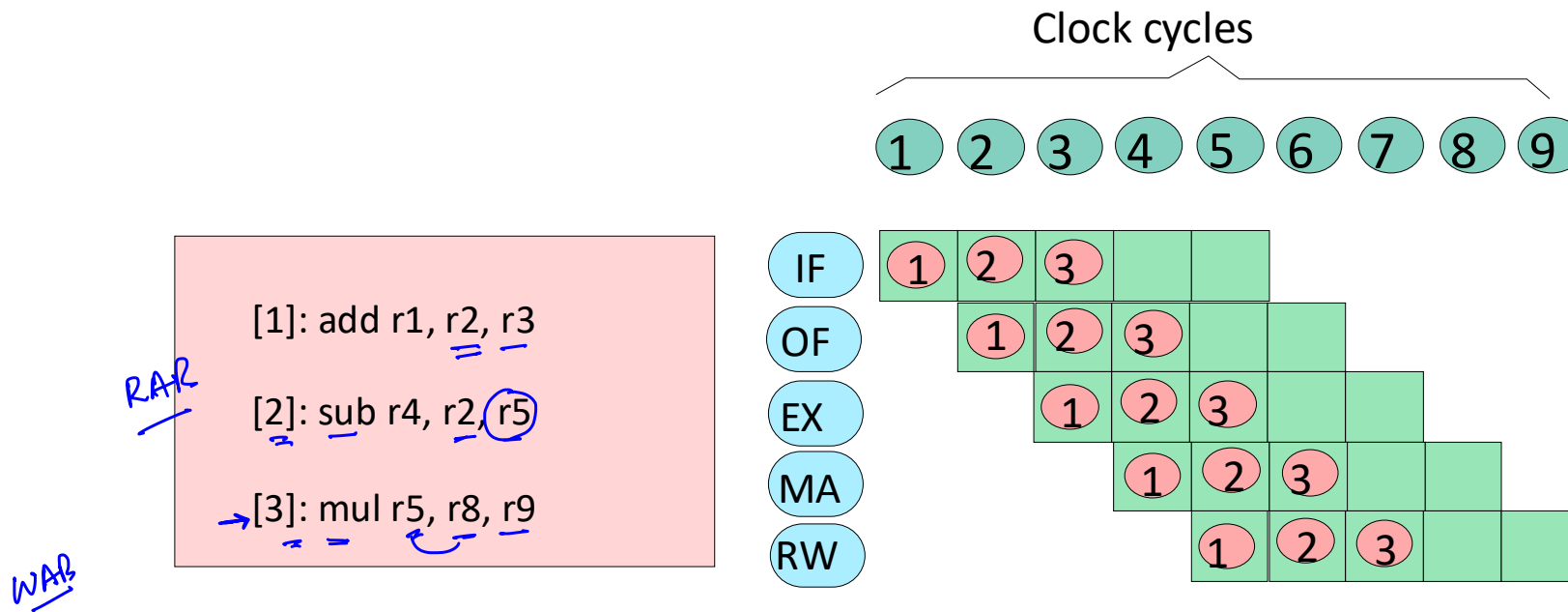
RAW



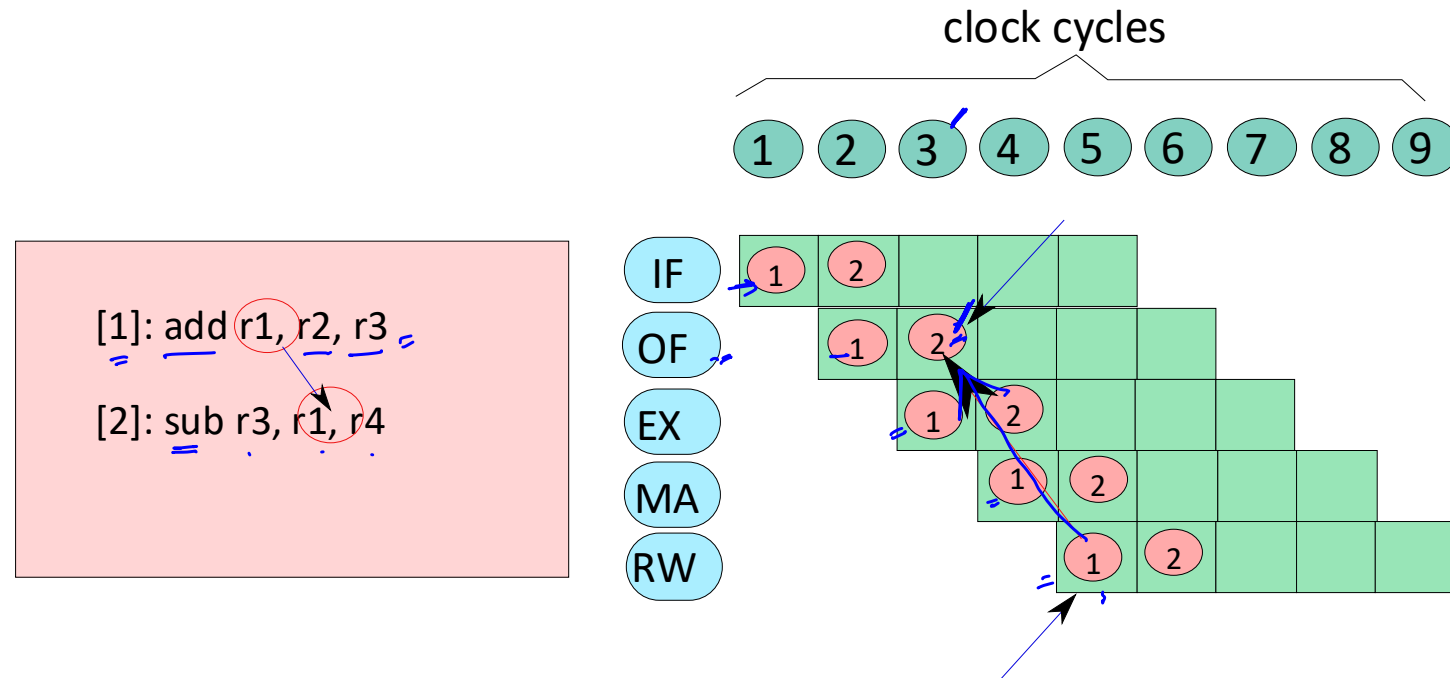
Rules for Constructing a Pipeline Diagram

- * It has 5 **rows**
 - * One per each stage
 - * The rows are **named** : IF, OF, EX, MA, and RW
- * Each **column** represents a clock cycle
- * Each **cell** represents the execution of an instruction in a stage
 - * It is **annotated** with the name(label) of the instruction
- * Instructions proceed from one stage to the next across clock cycles

Example



Data Hazards



RAW Ⓢ

* Instruction 2 will read incorrect values !!!

Data Hazard

Definition: A **hazard** is defined as the possibility of erroneous execution of an instruction in a pipeline. A data hazard represents the possibility of erroneous execution because of the unavailability of data, or the availability of **incorrect** data.

- * This situation represents a **data hazard**
- * In specific,
 - * it is a **RAW** (read after write) hazard
- * The earliest we can dispatch instruction 2, is cycle 5

Other Types of Data Hazards

- * Our pipeline is in-order

Definition: In an **in-order** pipeline (such as ours), a preceding instruction is always ahead of a succeeding instruction in the pipeline. Modern processors however use out-of-order pipelines that break this rule. It is possible for later instructions to execute before earlier instructions.

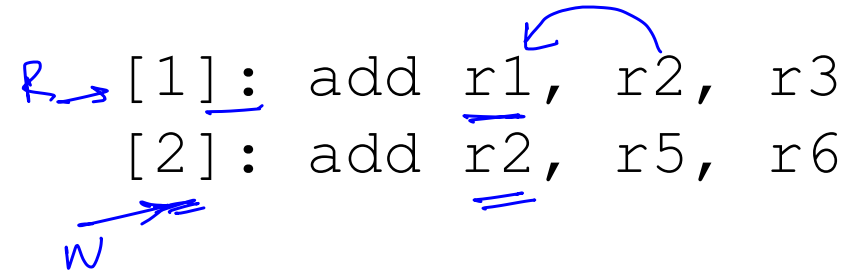
- * We will only have RAW hazards in our pipeline.
- * Out-of-order pipelines can have **WAR** and **WAW** hazards

WAW Hazards

```
✓ [1]: add r1, r2, r3  
✓ [2]: sub r1, r4, r3
```

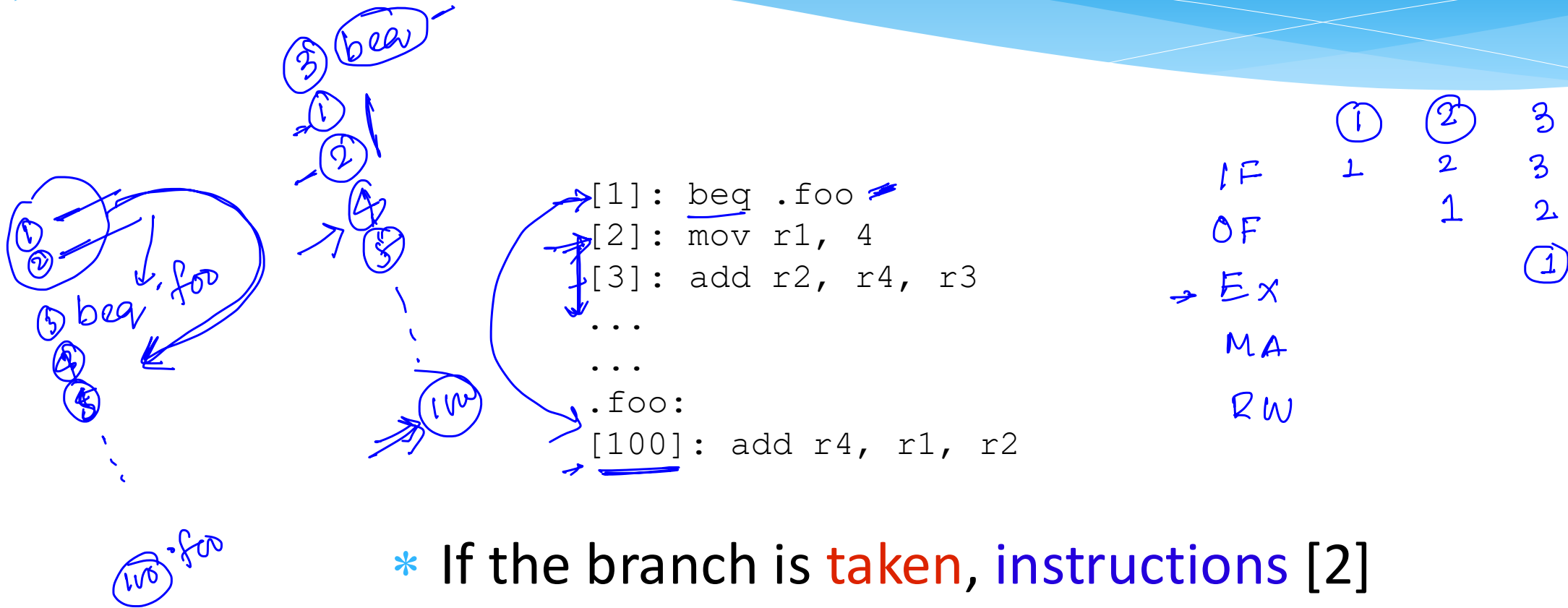
- * Instruction [2] cannot **write** the value of r1, before **instruction** [1] writes to it, will lead to a **WAW** hazard

WAR Hazards


[1]: add r1, r2, r3
[2]: add r2, r5, r6
W

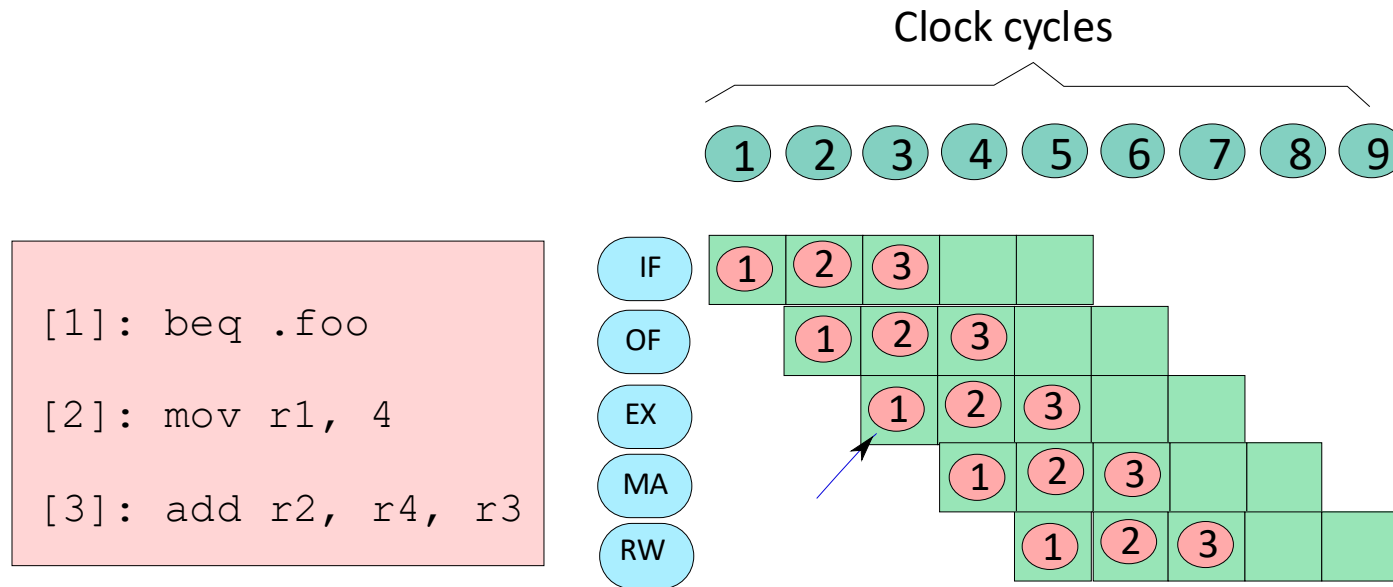
- * Instruction [2] cannot write the value of r2, before instruction [1] reads it → will lead to a WAR hazard

Control Hazards



- * If the branch is **taken**, instructions [2] and [3], might get fetched, incorrectly

Control Hazard – Pipeline Diagram



- * The two **instructions** fetched immediately after a branch instruction might have been fetched **incorrectly**.

Control Hazards

- * The two **instructions** fetched immediately after a branch instruction might have been fetched **incorrectly**.
- * These instructions are said to be on the **wrong path**
- * A **control hazard** represents the **possibility** of **erroneous** execution in a **pipeline** because instructions in the **wrong path** of a **branch** can possibly get **executed** and save their results in memory, or in the register file

Structural Hazards

- * A **structural hazard** may occur when two instructions have a conflict on the same set of resources in a cycle

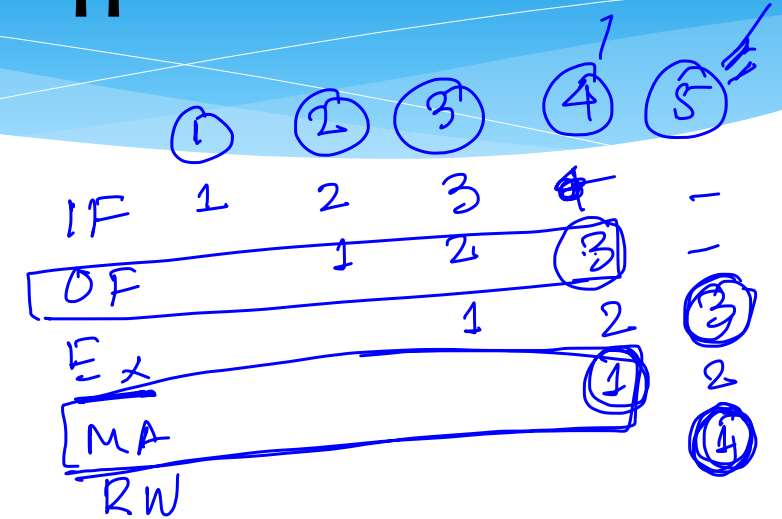
Example :

- * Assume that we have an add instruction that can read one operand from memory
- * `add r1, r2, 10[r3]`

add r1, r2, 10[r3]
add r1, r2, 10[r3]

Structural Hazards - II

- [1]: st r4, 20[r5]
- [2]: sub r8, r9, r10
- [3]: add r1, r2, 10[r3]

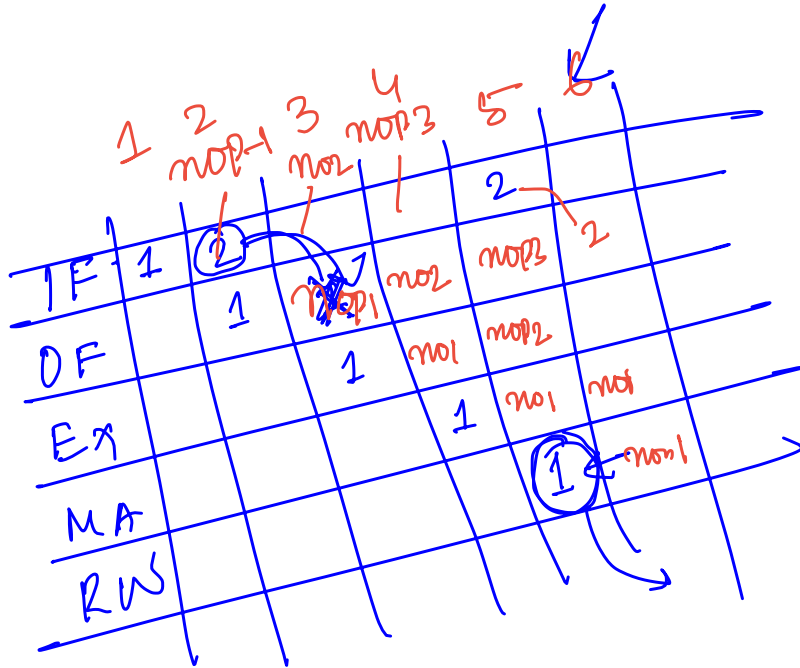


- * This code will have a structural hazard
 - * [3] tries to read 10[r3] (MA unit) in cycle 4
 - * [1] tries to write to 20[r5] (MA unit) in cycle 4
- * Does not happen in our pipeline

Solutions in Software

- * Data hazards

- * Insert **nop** instructions, reorder **code**

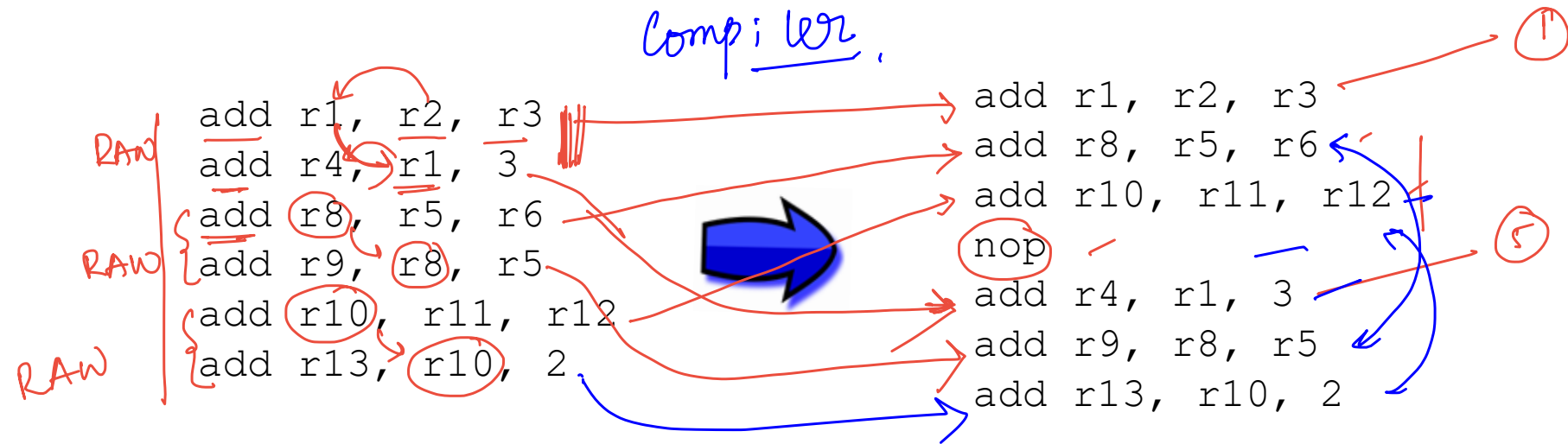


```
[1]: add r1, r2, r3
[2]: sub r3, r1, r4
```



```
[1]: add r1, r2, r3
[2]: nop
[3]: nop
[4]: nop
[5]: sub r3, r1, r4
```

Code Reordering



Control Hazards

- * **Trivial Solution** : Add two nop instructions after every branch
- * **Better solution** :
 - * Assume that the two instructions fetched after a branch are **valid** instructions
 - * These instructions are said to be in the delay slots
 - * Such a branch is known as a **delayed branch**

Example with 2 Delay Slots

```
- add r1, r2, r3  
- add r4, r5, r6  
→ b .foo  
add r8, r9, r10
```



```
→ b .foo  
→ add r1, r2, r3  
→ add r4, r5, r6  
→ add r8, r9, r10
```

- * The **compiler** transfers **instructions** before the branch to the **delay slots**.
- * If it cannot find 2 valid **instructions**, it inserts **nops**.